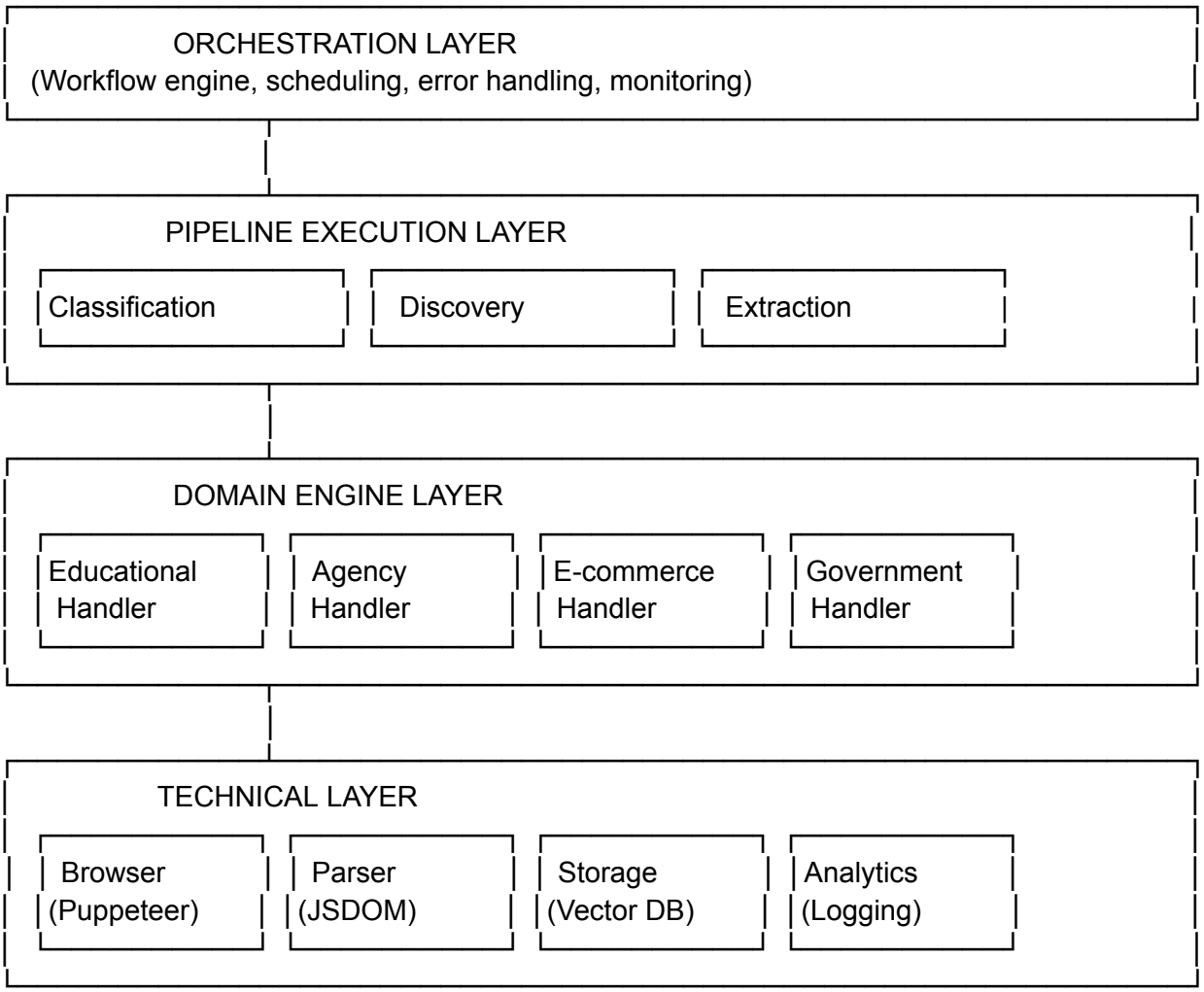
Enterprise-Grade Intelligent Scraping Architecture

Core Principles

1. **Modularity**: Each component is independent and replaceable
2. **Extensibility**: New domains, extractors, and rules can be added without refactoring
3. **Configuration-Driven**: Business logic lives in config files, not code
4. **Observable**: Every decision is logged and traceable
5. **Maintainable**: Organized by concern, not by technology

Architecture Overview



LAYER 1: CLASSIFICATION ENGINE

Purpose

Identify what type of website we're dealing with and load appropriate strategies.

Components

1. Fingerprinting Service

```
Input: Homepage URL
Output: {
  domain_type: 'educational' | 'agency' | 'ecommerce' | 'government' | 'unknown',
  confidence: 0.0-1.0,
  signals: {
    url_patterns: [...],
    metadata_signals: {...},
    content_signals: {...},
    structural_signals: {...}
  },
  cms_detected: 'wordpress' | 'drupal' | 'custom' | null,
  language: 'en',
  scale: 'small' | 'medium' | 'large'
}
```

Signals to Check:

- **URL patterns**: .edu, .gov, .com, /admission/, /products/, /schemes/
- **Metadata**: title, description, schema.org markup (JSON-LD)
- **Content**: Keyword frequency (education, agency, commerce, government terms)
- **Structure**: Navigation patterns, page layout
- **Robots.txt & Sitemap**: Available endpoints, crawl restrictions
- **Technology**: CMS detection (WordPress, Drupal, custom)

Extensibility: Add new signal detectors without modifying core logic

---

2. Strategy Loader

```
Input: domain_type
Output: {
  scraping_strategy: {...},
  extraction_rules: {...},
  content_prioritization: [...],
  update_frequency: 'hourly' | 'daily' | 'weekly' | 'monthly',
  storage_schema: {...}
}
```

Maps detected domain to configuration file.

LAYER 2: DISCOVERY ENGINE

Purpose

Intelligently identify which pages to scrape based on domain type, without generic crawling.

Components

1. Page Classification System

Each page gets classified into types:

Educational Domain:

```
PAGE_TYPES: {
  PROGRAM_PAGE: { patterns: [/program/, /course/, /bachelor/], priority: 10 },
  ADMISSION_PAGE: { patterns: [/admission/, /apply/, /enroll/], priority: 9 },
  CALENDAR_PAGE: { patterns: [/calendar/, /academic-timeline/, /dates/], priority: 10 },
  FACULTY_PAGE: { patterns: [/faculty/, /department/, /staff/], priority: 6 },
  STUDENT_SERVICES: { patterns: [/hostel/, /library/, /counseling/, /placement/], priority: 7 },
  NEWS_PAGE: { patterns: [/news/, /event/, /announcement/], priority: 3 },
  GENERAL_PAGE: { patterns: [] , priority: 2 }
}
```

Agency Domain:

```
PAGE_TYPES: {
  SERVICE_PAGE: { patterns: [/service/, /solution/], priority: 10 },
  CASE_STUDY: { patterns: [/portfolio/, /work/, /case-study/, /project/], priority: 9 },
  TEAM_PAGE: { patterns: [/team/, /about/, /leadership/, /people/], priority: 7 },
  EXPERTISE_PAGE: { patterns: [/expertise/, /skills/, /capabilities/], priority: 6 },
  PRICING_PAGE: { patterns: [/pricing/, /packages/, /plans/], priority: 5 },
  PROCESS_PAGE: { patterns: [/how-it-works/, /process/, /methodology/], priority: 6 },
  BLOG_PAGE: { patterns: [/blog/, /insights/, /resources/], priority: 2 },
  CONTACT_PAGE: { patterns: [/contact/, /reach-us/], priority: 3 }
}
```

E-Commerce Domain:

```
PAGE_TYPES: {
  CATEGORY_PAGE: { patterns: [/category/, /shop/, /collection/], priority: 10 },
  PRODUCT_PAGE: { patterns: [/product/, /item/, /sku/], priority: 9, limit: 50 },
  BRAND_PAGE: { patterns: [/about/, /brand/, /story/, /mission/], priority: 8 },
  POLICY_PAGE: { patterns: [/return/, /shipping/, /refund/, /warranty/], priority: 5 },
  REVIEW_PAGE: { patterns: [/reviews/, /testimonials/], priority: 3 }
}
```

Government Domain:

```
PAGE_TYPES: {
  SCHEME_PAGE: { patterns: [/scheme/, /program/, /subsidy/], priority: 10 },
  ELIGIBILITY_PAGE: { patterns: [/eligibility/, /criteria/, /benefits/], priority: 10 },
  PROCESS_PAGE: { patterns: [/application/, /process/, /how-to-apply/], priority: 10 },
  DOCUMENT_PAGE: { patterns: [/document/, /requirement/, /checklist/], priority: 9 },
  GRIEVANCE_PAGE: { patterns: [/grievance/, /complaint/, /redressal/], priority: 5 },
  ANNOUNCEMENT_PAGE: { patterns: [/announcement/, /notice/, /circular/], priority: 4 }
}
```

2. Discovery Strategy

```
DISCOVERY MODES:

A. SITEMAP-DRIVEN (Preferred)
   - Parse sitemap.xml
   - Filter URLs by page_type
   - Respect priority weights
   - Result: Clean list of URLs to scrape

B. NAVIGATION-DRIVEN (Fallback)
   - Extract main navigation links (header, footer, sidebar)
   - Classify each link by page_type
   - Follow up to 2 levels deep only
   - Result: Primary navigation pages only

C. RULES-BASED (Last resort)
   - Use URL patterns to identify page types
   - Crawl breadth-first with depth limit
   - Apply URL filtering rules
   - Result: Targeted page list

D. MIXED-HYBRID (Best for large sites)
   - Use sitemap + navigation + rules
   - Combine results, remove duplicates
   - Sort by priority and freshness
   - Apply per-domain limits
```

3. URL Pool & Queuing

```
DISCOVERY OUTPUT: {
  urls: [
    { url, page_type, priority, freshness, last_scraped },
    ...
  ],
  discovery_method: 'sitemap | navigation | rules | hybrid',
  total_identified: 150,
  prioritized_batch: [first 30 URLs sorted by priority],
  metadata: {
```

```
    sitemap_found: true,
    robots_restrictions: [...],
    estimated_total_pages: 200
  }
}
```

Extensibility: New discovery methods can be plugged in without affecting others

---

LAYER 3: EXTRACTION ENGINE

Purpose

Extract domain-specific, structured data from pages.

Components

1. Content Parser Framework

```
PARSER LAYERS:

Layer 1: DOM Structure Analysis
  - Remove noise (nav, footer, ads, modals)
  - Identify main content container
  - Extract headings hierarchy (H1→H2→H3)
  - Detect tables, lists, code blocks

Layer 2: Domain-Specific Extraction
  - Apply domain handlers (Educational, Agency, etc.)
  - Extract structured entities
  - Identify relationships between entities
  - Tag confidence scores

Layer 3: Semantic Enhancement
  - Entity linking (what is this about?)
  - Relationship extraction (how do things connect?)
  - Temporal data detection (dates, deadlines)
  - Classification scoring
```

2. Domain Handlers (Pluggable)

Educational Handler:

```
ENTITIES_TO_EXTRACT:
- Academic Programs
  { name, duration, eligibility, fees, admission_deadline }
- Departments
  { name, head, contact, programs_offered }
```

- Important Dates
  { event_name, date, type: 'admission|exam|registration|deadline' }
- Student Services
  { name, description, contact, availability }
- Faculty
  { name, title, department, contact, specialization }

RELATIONSHIP MAPPING:
- Program → Department
- Program → Admission Date
- Department → Faculty
- Faculty → Research Areas

Agency Handler:

ENTITIES_TO_EXTRACT:
- Services
  { name, description, industry_focus, methodology }
- Case Studies
  { title, industry, client_type, challenge, solution, results }
- Team Members
  { name, role, expertise, years_experience, linkedin }
- Process Steps
  { sequence, step_name, description, duration }
- Expertise Areas
  { area, description, level_of_expertise, examples }

RELATIONSHIP MAPPING:
- Service → Case Studies (which services were used)
- Service → Team Members (who specializes)
- Case Study → Results (measurable outcomes)

E-Commerce Handler:

ENTITIES_TO_EXTRACT:
- Product Categories
  { name, description, subcategories, featured_products }
- Product Information
  { name, sku, category, features, specifications, pricing_note }
- Brand Information
  { name, mission, values, story, founding_year }
- Policies
  { type: 'return|shipping|refund|warranty', description, terms }
- Reviews Aggregation
  { average_rating, total_reviews, top_themes_positive, top_themes_negative }

RELATIONSHIP MAPPING:
- Category → Subcategory → Product
- Product → Brand

- Product → Reviews (sentiment)
- Brand → Policy

Government Handler:

ENTITIES_TO_EXTRACT:
- Schemes
  { name, description, target_audience, benefits }
- Eligibility Criteria
  { scheme_id, criteria_items: [{field, condition, value}] }
- Application Process
  { scheme_id, steps: [{sequence, step_name, description}] }
- Required Documents
  { scheme_id, documents: [{name, type, required/optional}] }
- Important Dates
  { scheme_id, event_type, date, status: 'active|upcoming|closed' }
- Contact Information
  { office_name, address, phone, email, office_hours }
- Grievance Process
  { steps, contact_details, escalation_levels }

RELATIONSHIP MAPPING:
- Scheme → Eligibility
- Scheme → Process
- Scheme → Documents
- Scheme → Important Dates
- Process → Documents (when needed)

3. Structured Output Schema

```
EXTRACTION OUTPUT: {
  page: {
    url,
    page_type,
    title,
    description,
    last_updated,
    content_hash
  },
  entities: [
    {
      id: 'entity_type_uuid',
      type: 'program | service | scheme | product',
      data: { extracted fields },
      confidence: 0.95,
      raw_source_text: '...'
    }
  ],
  relationships: [
```

```
    {
      source_entity_id,
      target_entity_id,
      relationship_type: 'belongs_to | references | related_to',
      metadata: {}
    }
  ],
  metadata: {
    extraction_method: 'handler_name',
    processing_time_ms: 1500,
    quality_score: 0.92,
    errors: []
  }
}
```

Extensibility: New domain handlers inherit from base class, override entity extraction methods

---

LAYER 4: DOMAIN HANDLERS (Extensible Registry)

Structure

```
handlers/
├── base-handler.js          (Abstract base class)
├── educational-handler.js   (Concrete implementation)
├── agency-handler.js
├── ecommerce-handler.js
├── government-handler.js
└── custom-domain-handler.js (Template for new domains)

configs/
├── domains.config.json      (Domain definitions)
├── educational.config.json  (Domain-specific rules)
├── agency.config.json
├── ecommerce.config.json
├── government.config.json
└── custom-domain.config.json
```

Handler Base Class:

```
class DomainHandler {
  constructor(config) {
    this.config = config;
    this.entityTypes = {}; // Define in child class
    this.relationships = {}; // Define in child class
  }

  async extract(page) {
```

```
    // Child class implements
  }

  validateEntity(entity) {
    // Common validation logic
  }

  scoreConfidence(entity) {
    // Common confidence calculation
  }

  // Add more methods as needed
}
```

Adding a New Domain:

1. Create config file with page types, entities, relationships
2. Create handler extending base class
3. Register handler in handler registry
4. Test on sample sites
5. No changes to core architecture needed

LAYER 5: STORAGE & VECTORIZATION

Purpose

Store extracted data in formats suitable for retrieval and LLM integration.

Components

1. Multi-Format Storage

```
STORAGE DESTINATIONS:

A. Vector Database (Primary for RAG)
   Input: Extracted entities + text chunks
   Output: Embeddings + metadata

   Schema:
   {
     chunk_id: uuid,
     domain_type: string,
     page_type: string,
     entity_type: string,
     entity_id: uuid,
     text: string,
     embedding: float[],
     metadata: {
```

```
    url,
    entity_data,
    relationships,
    confidence_score,
    scraped_date,
    source_tags
  }
}
```

B. Relational Database (Entity & Relationship Store)
  Tables:
  - entities (id, type, domain, data_json, confidence)
  - relationships (source_id, target_id, type, metadata)
  - pages (url, domain_type, page_type, hash)

C. Document Store (JSON - Full page data backup)

```
{
  page_url,
  page_type,
  raw_extraction,
  timestamp
}
```

D. Knowledge Graph (Optional - for complex relationships)
  Nodes: entities
  Edges: relationships with types

2. Chunking Strategy (Per Domain)

EDUCATIONAL:
- Chunk by program/department
- Include context headers
- Link academic dates separately
- Example: "Computer Science Program [Dept: Engineering] - Duration: 4 years..."

AGENCY:
- Chunk by service or case study
- Keep client context with results
- Example: "Web Development Service [Industry: Tech] - Used in Project X..."

ECOMMERCE:
- Chunk by category + product group
- Aggregate reviews by theme
- Example: "Smartphones Category - Features: X, Y, Z - Positive feedback: durability..."

GOVERNMENT:
- Chunk by scheme
- Include eligibility + process in same chunk
- Example: "Scheme X - Eligibility: age 18+ - Process: Step 1, Step 2..."

3. Update Strategy

FRESHNESS LEVELS:

Tier 1: Critical (Update on every scrape)
- Government schemes, academic calendars, pricing changes

Tier 2: Important (Update daily)
- News, announcements, case studies

Tier 3: Reference (Update weekly)
- Team pages, service descriptions, brand info

Tier 4: Static (Update monthly or on-demand)
- About pages, policies, historical content

DIFFERENTIAL UPDATES:
- On re-scrape, compare content hashes
- Update only changed entities
- Preserve historical versions
- Maintain audit trail

LAYER 6: MONITORING & OBSERVABILITY

Components

1. Scraping Pipeline Monitoring

METRICS TO TRACK:

Per-Site:
- Pages discovered vs pages scraped
- Extraction success rate
- Average extraction confidence
- Time taken per page type
- Errors encountered

Per-Domain:
- Total entities extracted
- Entity distribution by type
- Relationship density
- Data freshness score
- Update frequency impact

Global:
- Resource utilization (CPU, memory, bandwidth)
- Cost per domain
- Success/failure rates

- Extraction quality trends

2. Quality Scoring

EXTRACTION QUALITY SCORE =
(Completeness × 0.3) +
(Accuracy × 0.3) +
(Relevance × 0.2) +
(Freshness × 0.2)

Completeness: Are all required fields extracted?
Accuracy: Do values match expected patterns?
Relevance: Is the data related to domain?
Freshness: How recent is the data?

3. Audit Trail

Every scrape records:
- Timestamp
- URL
- Page type detected
- Entities extracted (count, types)
- Confidence scores
- Changes made (new, updated, deleted)
- Errors encountered
- Processing time
- User who triggered (if manual)

LAYER 7: CONFIGURATION SYSTEM (Single Source of Truth)

Master Configuration Structure

```
{
  "domains": {
    "educational": {
      "identifier": "educational",
      "aliases": ["college", "university", "school"],
      "fingerprint": {
        "url_patterns": [".edu", ".ac.", "university", "college"],
        "metadata_keywords": ["university", "college", "academic"],
        "confidence_threshold": 0.8
      },
      "discovery": {
        "methods": ["sitemap", "navigation", "rules"],
        "max_pages": 200,
        "page_types": {
```

```
        "program": { "priority": 10, "patterns": [...] },
        "admission": { "priority": 9, "patterns": [...] }
      }
    },
    "extraction": {
      "handler": "EducationalHandler",
      "entities": [
        {
          "name": "AcademicProgram",
          "fields": [
            { "name": "program_name", "type": "string", "required": true },
            { "name": "duration_years", "type": "number", "required": true },
            { "name": "admission_deadline", "type": "date", "required": false }
          ]
        }
      ],
      "relationships": [
        {
          "source": "Program",
          "target": "Department",
          "type": "belongs_to"
        }
      ]
    },
    "update_schedule": {
      "frequency": "weekly",
      "tier_1_frequency": "daily",
      "tier_1_entities": ["calendar", "announcement"]
    },
    "validation": {
      "required_entities_min": 5,
      "min_confidence": 0.85,
      "schema_validation": true
    }
  },
  "agency": { /* similar structure */ },
  "ecommerce": { /* similar structure */ },
  "government": { /* similar structure */ }
 },
 "extraction_rules": {
  "date_formats": ["YYYY-MM-DD", "DD/MM/YYYY"],
  "currency_symbols": ["₹", "$", "€"],
  "noise_selectors": ["nav", "footer", ".ad", ".cookie"]
 }
}
```

EXTENSIBILITY PATTERNS

Adding a New Domain

Step 1: Create Configuration

configs/healthcare.config.json

Step 2: Create Handler

```
// handlers/healthcare-handler.js
class HealthcareHandler extends DomainHandler {
  constructor(config) {
    super(config);
    this.entityTypes = ['Hospital', 'Department', 'Doctor', 'Service'];
  }

  async extract(page) {
    // Custom extraction logic
  }
}
```

Step 3: Register Handler

```
// handlers/handler-registry.js
registry.register('healthcare', HealthcareHandler);
```

No core code changes needed.

Adding New Extraction Rules

Step 1: Update config

```
{
  "extraction_rules": {
    "custom_pattern": "..."
  }
}
```

Step 2: Add to base handler

```
async applyCustomPattern(text) {
  // Rule implementation
}
```

No pipeline code changes needed.

Adding New Storage Backend

Step 1: Implement storage interface

```
class CustomStorage implements StorageBackend {
  async save(data) { /* ... */ }
  async query(filter) { /* ... */ }
  async update(id, data) { /* ... */ }
}
```

Step 2: Register

```
storageRegistry.register('custom', CustomStorage);
```

No extraction logic changes needed.

Data Flow Example: Educational Website

```
URL: university.edu
↓
[CLASSIFICATION]
  → Fingerprint: .edu + "university" + campus structure
  → Result: domain_type = "educational" (0.98 confidence)
↓
[LOAD CONFIG]
  → Load educational.config.json
  → Load EducationalHandler
↓
[DISCOVERY]
  → Parse sitemap.xml
  → Identify pages: programs (20), admissions (5), dates (1)
  → Queue: 26 high-priority pages
↓
[EXTRACTION] - For each page:
  → Remove nav/footer/ads
  → Apply EducationalHandler
  → Extract: Program{ name, duration, fees, deadline }
  → Extract: Dates{ event, date, type }
  → Score confidence: 0.92
↓
[STORAGE]
  → Vector DB: chunk program description + metadata
  → Relational DB: Store Program entity + relationship to Department
  → Audit: Log extraction results
↓
[VECTOR EMBEDDING]
  → "Computer Science Program - 4 years - Deadline: June 30"
  → Embed + store with metadata
↓
```

[RAG CHATBOT]
User: "What are the admission deadlines?"
→ Retrieves relevant chunks from vector DB
→ LLM generates: "Admissions close on June 30 for all programs"

Key Advantages of This Architecture

✅ **Scalable**: Add new domains without rewriting core logic ✅ **Maintainable**: Configuration-driven, clear separation of concerns ✅ **Observable**: Full audit trail and quality metrics ✅ **Flexible**: Swap components (storage, handlers, parsers) ✅ **Testable**: Each layer can be tested independently ✅ **Future-proof**: New requirements plugged in as extensions ✅ **Domain-aware**: Each domain gets intelligent treatment ✅ **Efficient**: Only scrapes what matters for that domain ✅ **Traceable**: Every decision is logged and documented